
SYSTEM REQUIREMENTS - TEST

Apricart Pvt Ltd



1 Product Module

1.1 Product Entity

- **Fields:**

- **productId:** Unique identifier for the product.
- **productName:** Name of the product.
- **description:** Description of the product.
- **price:** Price of the product.
- **brand:** Brand of the product.
- **status:** Boolean indicating the availability status of the product.
- **warehouseId:** Warehouse ID where the product is stored.

1.2 ProductService

- **Methods:**

- **createProduct:** Method to create a new product entry.
- **updateProduct:** Method to update an existing product entry.
- **deleteProduct:** Method to delete a product entry.
- **getProductById:** Method to retrieve a product entry by its ID.

2 Cart Module Requirements

2.1 Cart Entity

- **Fields:**

- **cartId:** Unique identifier for the cart.
- **userId:** User ID associated with the cart. Use hardcoded (User@123)
- **productId:** Product ID added to the cart.
- **quantity:** Quantity of the product in the cart.
- **warehouseId:** Warehouse ID where the product is stored.

2.2 CartService

- **Methods:**

- **addToCart:** Method to add a product to the user's cart.
- **updateCart:** Method to update the quantity of a product in the cart.
- **removeFromCart:** Method to remove a product from the cart.
- **clearCart:** Method to clear all items from the cart.
- **getCartItems:** Method to retrieve all items in the user's cart.
- **getCartTotal:** Method to calculate the total price of items in the cart.

3 Order Module

3.1 Order Entity

- **Fields:**
 - `orderId`: Unique identifier for the order.
 - `userId`: User ID associated with the order.
 - `productId`: Product ID included in the order.
 - `quantity`: Quantity of the product in the order.
 - `totalPrice`: Total price of the order.
 - `status`: Status of the order (e.g., pending, shipped, delivered).
 - `warehouseId`: Warehouse ID where the product is stored.

3.2 OrderService

- **Methods:**
 - `placeOrder`: Method to place a new order.
 - `updateOrderStatus`: Method to update the status of an existing order.
 - `getOrderByUserId`: Method to retrieve orders associated with a user.
 - `getOrderById`: Method to retrieve an order by its ID.
 - `cancelOrder`: Method to cancel an existing order.
 - `getOrderTotal`: Method to calculate the total price of an order.

4 Coupon Module

4.1 Coupon Entity

- **Fields:**
 - `couponId`: Unique identifier for the coupon.
 - `couponCode`: Coupon code.
 - `discountAmount`: Discount amount provided by the coupon.
 - `minOrderAmount`: Minimum order amount required to use the coupon.
 - `city`: City where the coupon is applicable.
 - `warehouseId`: Warehouse ID where the coupon is applicable.
 - `isActive`: Boolean indicating whether the coupon is active.

4.2 CouponService

- **Methods:**
 - `validateCoupon`: Method to validate a coupon code.
 - `applyCoupon`: Method to apply a coupon discount to the order.

5 Warehouse Module

5.1 Warehouse Entity

- **Fields:**
 - `warehouseId`: Unique identifier for the warehouse.
 - `warehouseName`: Name of the warehouse.
 - `location`: Location of the warehouse.
 - `capacity`: Maximum capacity of the warehouse.
 - `availableCapacity`: Remaining available capacity of the warehouse.

5.2 WarehouseService

- **Methods:**
 - `createWarehouse`: Method to create a new warehouse.
 - `updateWarehouse`: Method to update an existing warehouse.
 - `deleteWarehouse`: Method to delete a warehouse.
 - `getWarehouseById`: Method to retrieve a warehouse by its ID.
 - `getAllWarehouses`: Method to retrieve all warehouses.

6 Checkout Module

6.1 CheckoutService

- **Methods:**
 - `processCheckout`: Method to process the checkout request.
 - `calculateTotal`: Method to calculate the total amount payable during checkout.
 - Other methods as needed for payment processing, order placement, and confirmation.

7 Coupon Module Requirements

7.1 Coupon Validation and Application

- **Validate the coupon code** provided by the user to ensure it's valid and applicable.
- **Apply the coupon discount** to the total amount payable if the coupon is valid and meets the criteria (e.g., minimum order amount, usage limits).
- **Handle different types of coupons** (e.g., percentage-based discounts, fixed amount discounts).

8 Checkout Module Requirements

8.1 Checkout Process

- **Initiate the checkout process** when the user submits the order.
- **Calculate the total amount payable** based on the subtotal, taxes, and any discounts applied (including coupon discounts).
- **Process payment** using the selected payment method (e.g., cash, online payment).
- **Generate a unique order ID** for each transaction.
- **Update the order status and payment status** accordingly.
- **Send notifications and confirmation emails** to the user regarding the order details and payment status.

9 Error Handling

- **Handle errors gracefully** and provide informative error messages to the user.
- **Log errors and exceptions** for debugging and troubleshooting purposes.
- **Ensure data consistency and integrity** throughout the checkout process.

10 Other Requirements

- Use `@JsonInclude(JsonInclude.Include.NON_NULL)` annotation to exclude null values from JSON serialization.
- Handle exceptions appropriately and return meaningful error responses.
- Implement necessary business logic for coupon validation, order processing, and payment handling.
- Ensure proper logging of errors and exceptions for debugging purposes.
- Write unit test cases.
- Maintain a postman collection.

1. Database Setup:

- Choose a database (e.g., MySQL, PostgreSQL) for storing data related to products, carts, orders, coupons, warehouses, etc.
- Set up the database schema with tables corresponding to the entities defined in the requirements.

2. Spring Boot Project Setup:

- Create a new Spring Boot project using Spring Initializr or your preferred method.

- Add dependencies for Spring Web, Spring Data JPA, and the chosen database driver.
3. **Entity Classes:**
 - Create entity classes for Product, Cart, Order, Coupon, Warehouse, etc., with appropriate annotations (`@Entity`, `@Table`, `@Id`, `@ManyToOne`, etc.) to map them to database tables.
 4. **Repository Interfaces:**
 - Define repository interfaces extending `JpaRepository` for each entity to handle database operations (e.g., CRUD operations).
 5. **Service Classes:**
 - Implement service classes for each module (e.g., `ProductService`, `CartService`, `OrderService`, `CouponService`, `WarehouseService`) to encapsulate business logic and interact with repositories.
 6. **Controller Classes:**
 - Create REST controller classes to define endpoints for each module and handle HTTP requests (GET, POST, PUT, DELETE).
 - Inject service classes into controllers to delegate request processing.
 7. **DTO (Data Transfer Object) Classes:**
 - Define DTO classes to represent data transferred between the client and the server. Use these classes to serialize/deserialize JSON objects.
 - Include fields for request/response payloads, ensuring they match the API contract.
 8. **Coupon Validation and Application Logic:**
 - Implement coupon validation and application logic in the `CouponService` class.
 - Validate coupon codes, check criteria (e.g., minimum order amount), and apply discounts as needed.
 9. **Checkout Process:**
 - Implement the checkout process in the `CheckoutService` class.
 - Calculate the total amount payable, process payment, generate order IDs, update order/payment status, and coupon.
 10. **Exception Handling:**
 - Implement global exception handling using `@ControllerAdvice` to catch and handle exceptions gracefully.
 - Return appropriate error responses with meaningful error messages.
 11. **Logging:**
 - Integrate logging frameworks like Logback or Log4j2 to log errors, exceptions, and other relevant information.
 - Log data consistently across the application for debugging and troubleshooting.

12. Testing - Optional:

- Write unit tests and integration tests using JUnit to ensure the correctness of business logic and API endpoints.
- Test coupon validation, checkout process, error handling, and other functionalities.

13. Caching - Optional:

- Set up Redis server or use a managed Redis service.
- Configure connection properties such as host, port, and authentication credentials.
- Use Redis cache in spring boot

14. Deployment:

- Deploy the Spring Boot application to a server (e.g., Tomcat, Jetty).
- Configure the application properties for the chosen database connection.

15. Documentation and GitHub:

- Document the project structure, API endpoints, and usage instructions.
- Upload the code to GitHub or another version control system and share the repository link top review.